Episodio 4 de 14: La solución del paper: el taco de vales

Serie «Leer sin que te quiten el libro»: la réplica del paper «A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting» (Euro-Par 2024), explicada para principiantes.

- **Episodio anterior:** episodio 3, «Los rivales: candados, cuentas dobles y basura retrasada».
- **Episodio siguiente:** episodio 5, «Cómo se hizo la réplica, y cuánto fiarse».

Lo esencial de este episodio:

- El tablón guarda un taco de vales prepagados. Arrancar un vale lee la dirección y reserva el libro en un solo gesto: un fetch_add que nunca falla.
- La regla de oro: libreta = vales en el taco + vales en manos de la gente.
- Por eso la lectura es wait-free: termina siempre en un número acotado de pasos, y recargar el taco reintenta como mucho P veces.

3.6 Autores (el algoritmo del paper)

Idea en una frase: el tablón guarda un taco de vales prepagados, y arrancar un vale te da la dirección del libro en un solo gesto indivisible.

Esta es la parte central, así que va con calma y de varias formas.

La historia de los vales

El bibliotecario pone el libro 42 en el tablón, y **antes** anota en la libreta del libro: «hay 100 usuarios». Esos 100 son imaginarios: son 100 vales que deja en un taco pegado al tablón.

Cuando Marta quiere el libro, **arranca un vale**. El vale lleva impreso «estante 42». En un único gesto ha hecho dos cosas: ha leído dónde está el libro y se ha llevado una reserva pagada. Ya no hay momento peligroso, porque no hay dos pasos.

Fíjate en lo bonito: Marta **ni ha tocado la libreta**. El libro ya contaba con ella.

Cuando Beto cambia el tablón, se lleva el taco con los vales que queden, digamos 97. Va a la libreta y resta esos 97, porque nadie los usará. Quedan 3: justo los 3 vales que hay en manos de la gente. El libro sigue vivo. Cada vez que alguien termina, resta 1. El último deja la libreta en 0 y lo tira **en ese mismo instante**. Sin retrasos, sin caja de pendientes.

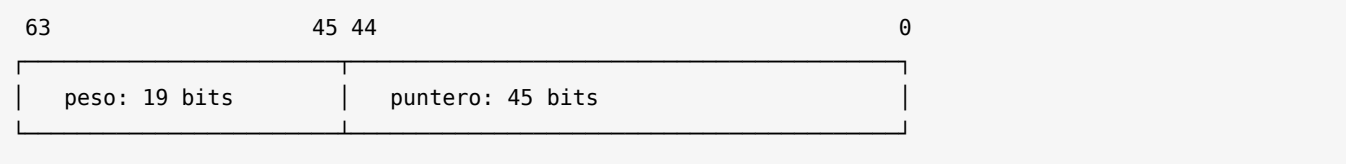
La regla de oro

> **Libreta = vales en el taco + vales en manos de la gente.**

Mientras esa igualdad se cumpla, nunca puede llegar a 0 algo que alguien pueda usar. En el paper, a cada vale se le llama **peso** (*weight*), y al sistema, **conteo de referencias ponderado**.

Cómo cabe todo en una palabra

Para que arrancar el vale y leer la dirección sean un solo gesto, la dirección y la cantidad de vales deben estar en la **misma palabra de 64 bits**:



Los punteros reales usan 48 bits, y los 3 últimos siempre son 0 por alineación. Eso deja 19 bits libres para el peso: hasta **C = 2¹⁹ − 1 = 524 287** vales.

Ejemplo completo con números pequeños

Tomamos C = 10 vales como máximo y P = 3 hilos como máximo (cumple la regla C ≥ 2P, que luego explicamos).

Paso	Quién	Acción	Tablón	Libreta de A	Libreta de B
0		situación inicial	(A, 10)	10	
1	Marta	`load`: `fetch_sub` del peso	(A, 9)	10	
2	Luis	`load`: `fetch_sub` del peso	(A, 8)	10	
3	Beto	prepara B: suma C a su libreta	(A, 8)	10	10
4	Beto	`exchange`: pone B y se lleva (A, 8)	(B, 10)	10	10
5	Beto	resta los 8 vales sobrantes a A	(B, 10)	**2**	10
6	Marta	termina: resta 1	(B, 10)	1	10
7	Luis	termina: resta 1	(B, 10)	**0 → destruido**	10

Comprueba la regla de oro en el paso 2: 8 en el taco + 2 en mano = 10. ✔️ Y en el paso 5: 0 en el taco + 2 en mano = 2. ✔️

Compáralo con la carrera de la Parte 1. Ahí Marta leía y luego sumaba, y en medio Beto destruía. Aquí Marta no tiene un «luego»: su reserva ya está hecha.

`load` y `store`, paso a paso

`load`:

1. `fetch_sub`: quitar 1 al peso. Devuelve puntero y peso anterior. Nunca falla.
2. Si el peso sigue siendo suficiente (≥ P), **fin**. Es casi siempre: un único paso atómico.
3. Si bajó de P, recargar llamando a `increase_weight` (ver abajo).

`store`:

1. Sumar C − 1 a la libreta del objeto nuevo.
2. `exchange`: poner (nuevo, C) y quedarse con (viejo, peso).
3. Restar `peso` a la libreta del viejo. Si llega a 0, se destruye ya.

Tres pasos, sin bucles. Por eso es **wait-free** y de tiempo constante.

La recarga, y por qué es la verdadera aportación

El taco se agota. Antes de eso hay que recargar. Si la recarga pudiera reintentarse para siempre, el algoritmo no sería wait-free. Aquí está la novedad del paper.

Cuando el peso cae por debajo de P, el hilo llama a `increase_weight`:

1. Suma a la libreta los vales que va a añadir: C − peso.
2. Intenta con un `CAS` cambiar (A, peso) por (A, peso + C − peso_inicial).
3. Si funciona, listo.
4. Si falla, mira **por qué** falló:

Lo que ve al fallar	Qué significa	Qué hace
El puntero ya es otro	Beto cambió el tablón: recargar ya no sirve	Deshace la suma y sale
El peso **subió**	Otro hilo ya recargó	Deshace la suma y sale
El peso **bajó**	Otro hilo arrancó un vale	Reintenta con el peso nuevo

La pregunta es: ¿cuántas veces se puede repetir? Solo reintenta si el peso bajó. Y el peso empezó por debajo de P y nunca baja de 0. Por tanto, reintenta **como mucho P veces**. Ahí está la cota, sin depender de la suerte.

Algoritmos anteriores salían solo si el `CAS` funcionaba o si el peso era suficiente. El truco nuevo es salir **también en cuanto se detecta que otro ya recargó**, aunque el peso vuelva a estar bajo en ese momento.

Por qué nunca se acaban los vales (las dos reglas)

- **R-1: C ≥ 2P.** El taco completo tiene al menos el doble de vales que hilos.
- **R-2: el peso solo cambia de dos formas:** bajando de 1 en 1 o saltando a un valor mayor que P.

Dicho con personas: cada hilo solo puede arrancar **un vale de más** antes de ponerse a recargar, porque no puede hacer otro `load` hasta que termine. Con P hilos como máximo, el peso baja como mucho P por debajo del umbral P. Y P − P = 0. Nunca es negativo. Por eso el umbral es precisamente P: deja sitio para que todos los hilos a la vez se pasen una vez.

Garantía	**Wait-free** en `load` y `store`; lock-free en `CAS`
Memoria extra	**Ninguna**; libera al instante
A favor	Una sola operación por lectura, que nunca falla. Y `store` en tiempo constante
En contra	Leer **modifica** el tablón, así que un `CAS` puede fallar por culpa de un simple lector. Por eso sufre con muchas actualizaciones sobre pocas pilas

Una nota sobre versiones. El binario oficial no usa exactamente el código que se ha explicado aquí. El artefacto trae una versión anterior en la que la cuenta **sube** (`tag`, `fetch_add`) en vez de bajar. La del repositorio del equipo es la que **baja** (`weight`, `fetch_sub`). Son el mismo algoritmo en espejo, como contar los vales usados en vez de los que quedan. En las dos, el camino rápido es una sola instrucción atómica.